

深入MCP通信方式

三种MCP通讯方式

MCP (Model Context Protocol) 协议目前支持**三种主要通信方式**，分别是：

stdio (标准输入输出)

- **工作原理：**

通过本地进程的标准输入 (stdin) 和标准输出 (stdout) 进行通信。客户端以子进程的形式启动MCP服务器，双方通过管道交换JSON-RPC格式的消息，消息以换行符分隔。

- **适用场景：**

- 本地进程间通信（如命令行工具、文件系统操作）。
- 简单的批处理任务或工具调用。

- **优点：**

- 实现简单，低延迟。
- 无需网络配置，适合本地开发。

- **限制：**

- 仅限本地使用，不支持分布式部署。
- 服务端不能输出控制台日志（会污染协议流）。

SSE (Server-Sent Events)

- **工作原理：**

基于HTTP长连接实现服务器到客户端的单向消息推送。客户端通过 `GET /sse` 建立长连接，服务器通过SSE流发送JSON-RPC消息；客户端通过 `POST /message` 发送请求或响应。

- **适用场景：**

- 远程服务调用（如云服务、多客户端监控）。
- 需要实时数据推送的场景（如对话式AI的流式输出）。

- **优点：**

- 支持实时单向推送，适合流式交互。

- **限制：**

- **已逐步被弃用**（2025年3月后被Streamable HTTP取代）。

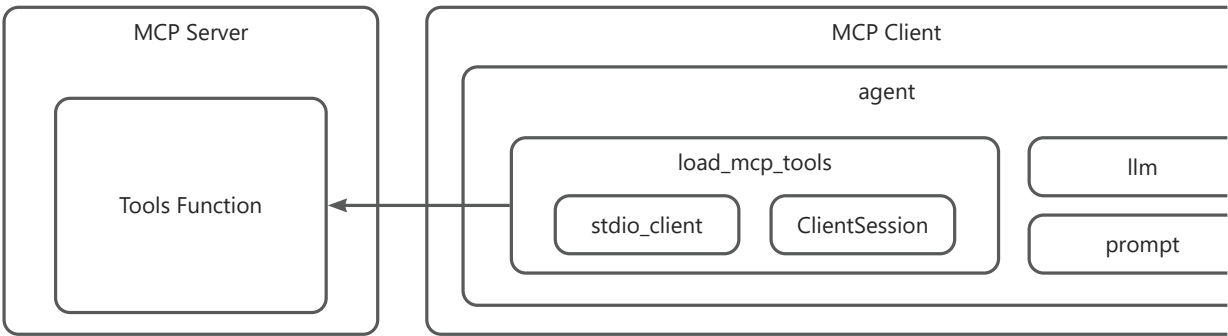
- 连接中断后无法恢复，需重新建立。
- 服务器需维持长连接，资源消耗较高。

Streamable HTTP（流式HTTP）

- **工作原理：**
2025年3月引入的新传输方式，替代了SSE。通过统一的 `/message` 端点实现双向通信，支持以下特性：
 - 客户端通过HTTP POST发送请求（如工具调用）。
 - 服务器可将响应升级为SSE流式传输（当需要时）。
 - 支持无状态模式（Stateless Server），无需维持长连接。
- **适用场景：**
 - 高并发远程服务调用。
 - 需要灵活流式响应的场景（如AI助手的动态输出）。
- **优点：**
 - **解决SSE的缺陷：**
 - 支持连接恢复（无需重新开始）。
 - 无需服务器维持长连接，降低资源压力。
 - 统一端点（ `/message` ），简化接口设计。
 - 兼容基础设施（如中间件、负载均衡）。
- **推荐使用：**
当前MCP官方推荐的传输方式，尤其适合生产环境和云服务。

实现基于stdio的mcp服务

stdio模式mcp服务架构：



第一步：创建mcp server（包含工具能力）

使用 FastMCP 构建 mcp server:

Python | 运行代码 复制代码

```
1  from mcp.server.fastmcp import FastMCP
2
3  mcp = FastMCP("Math Tools")
4
5  @mcp.tool()
6  def add(a: int, b: int) -> int:
7      """Add two numbers"""
8      return a + b
9
10 @mcp.tool()
11 def multiply(a: int, b: int) -> int:
12     """Multiply two numbers"""
13     return a * b
14
15 if __name__ == "__main__":
16     mcp.run(transport="stdio")
```

以上代码包含两部分内容：

1. 使用 `@mcp.tool()` 装饰器注册的 mcp 工具方法；
2. 使用 `mcp.run(transport="stdio")` 启动 stdio mcp 服务。

第二步：启动mcp server

找到 mcp server 所在文件夹，使用 python 命令启动服务（相当于启动了对 IO 流中 read 和 write 事件的监听）：

Bash | 运行代码 复制代码

```
1  $ python app/fastmcp/stdio/math_tools.py
```

第三步：开发mcp client（包含智能体）

这里又分为三小步：

3.1 定义 stdio server 参数

▼

Python | [运行代码](#) [复制代码](#)

```
1 server_params = StdioServerParameters(  
2     command="python",  
3     args=["/Users/sam/Xiaoluyy/ai/agent_test/app/fastmcp/math_tools.py"],  
4 )
```

3.2 读取 stdio mcp tools

▼

Python | [运行代码](#) [复制代码](#)

```
1 async with stdio_client(server_params) as (read, write):  
2     async with ClientSession(read, write) as session:  
3         await session.initialize()  
4         tools = await load_mcp_tools(session) # 自动加载MCP服务器提供的工具
```

3.3 定义智能体，加载 mcp tools

▼

Python | [运行代码](#) [复制代码](#)

```
1 agent = create_react_agent(llm, tools) # 创建React Agent  
2 response = await agent.ainvoke(input={"messages": [("user", "what's (3 + 5) x
```

mcp客户端完整源码

```
1 import asyncio
2
3 from mcp import ClientSession, StdioServerParameters
4 from mcp.client.stdio import stdio_client
5 from langchain_mcp_adapters.tools import load_mcp_tools
6 from langgraph.prebuilt import create_react_agent
7
8 from app.common import llm
9
10 # 使用stdio模式
11 server_params = StdioServerParameters(
12     command="python",
13     args=["/Users/sam/Xiaoluyy/ai/agent_test/app/fastmcp/math_tools.py"],
14 )
15
16 async def main():
17     async with stdio_client(server_params) as (read, write):
18         async with ClientSession(read, write) as session:
19             await session.initialize()
20             tools = await load_mcp_tools(session) # 自动加载MCP服务器提供的工具
21             print(tools)
22             agent = create_react_agent(llm, tools) # 创建React Agent
23             response = await agent.ainvoke(input={"messages": [("user", "what")
24             print(response)
25
```

第四步：启动智能体

```
1 asyncio.run(main())
```

运行结果：

```

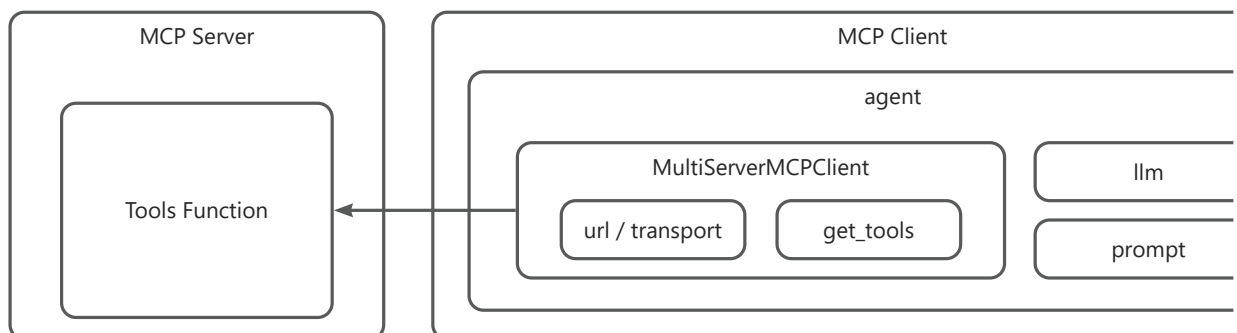
1  {'messages': [
2      HumanMessage(content="what's (3 + 5) x 12?", additional_kwargs={}, response
        _metadata={}, id='baefbd02-0e6f-4238-aac0-0989852ea097'),
3      AIMessage(content='', additional_kwargs={'tool_calls': [{'index': 0, 'id':
        'call_3f383dc314fe4b0ebc70c8', 'function': {'arguments': '{"a": 3, "b": 5}',
        'name': 'add'}, 'type': 'function'}]}, response_metadata={'finish_reason': 't
        ool_calls', 'model_name': 'qwen3-235b-a22b'}, id='run--7477b051-f147-4772-be2
        7-fc443813e5e4-0', tool_calls=[{'name': 'add', 'args': {'a': 3, 'b': 5}, 'i
        d': 'call_3f383dc314fe4b0ebc70c8', 'type': 'tool_call'}])),
4      ToolMessage(content='8', name='add', id='569efaef-c880-4b96-8a76-ecb1184da5
        37', tool_call_id='call_3f383dc314fe4b0ebc70c8'),
5      AIMessage(content='', additional_kwargs={'tool_calls': [{'index': 0, 'id':
        'call_287db3d531af4d2d82adba', 'function': {'arguments': '{"a": 8, "b": 1
        2}', 'name': 'multiply'}, 'type': 'function'}]}, response_metadata={'finish_r
        eason': 'tool_calls', 'model_name': 'qwen3-235b-a22b'}, id='run--506ce062-0dc
        c-4217-bcca-0e1be306ad0c-0', tool_calls=[{'name': 'multiply', 'args': {'a':
        8, 'b': 12}, 'id': 'call_287db3d531af4d2d82adba', 'type': 'tool_call'}])),
6      ToolMessage(content='96', name='multiply', id='69357610-bd4b-4486-8192-b87a
        3d35d6eb', tool_call_id='call_287db3d531af4d2d82adba'),
7      AIMessage(content='The result of  $(3 + 5) \times 12$  is  $\boxed{96}$ .', a
        dditional_kwargs={}, response_metadata={'finish_reason': 'stop', 'model_nam
        e': 'qwen3-235b-a22b'}, id='run--428dda6d-ebd6-4efd-beb4-b93e7c595ecb-0')
8  ]}

```

实现基于sse的mcp服务

注意：sse已被官方废弃，优先使用streamable-http，两者从代码层面来看，差异不大

sse/streamable-http模式mcp服务架构：



第一步：创建mcp server端

Python

运行代码 复制代码

```
1 if __name__ == "__main__":
2     mcp.run(transport="sse")
```

第二步：启动mcp server

Bash

运行代码 复制代码

```
1 $ python app/fastmcp/sse/math_tools.py
```

第三步：开发mcp client

与 stdio 差异点，其余相同：

Python

运行代码 复制代码

```
1 from langchain_mcp_adapters.client import MultiServerMCPClient
2
3 client = MultiServerMCPClient(
4     {
5         "math": {
6             "url": "http://127.0.0.1:8000/sse",
7             "transport": "sse",
8         }
9     }
10 )
11 tools = await client.get_tools()
```

第四步：启动智能体

Python

运行代码 复制代码

```
1 asyncio.run(main())
```

实现基于streamable_http的mcp服务

启动服务：

Python

运行代码 复制代码

```
1 mcp.run(transport="streamable-http")
```

客户端连接服务：

Python | 运行代码 复制代码

```
1 client = MultiServerMCPCClient(  
2     {  
3         "math": {  
4             "url": "http://127.0.0.1:8000/mcp",  
5             "transport": "streamable_http",  
6         }  
7     }  
8 )  
9 tools = await client.get_tools()
```

附

python with 语法

Python | 运行代码 复制代码

```
1 from contextlib import contextmanager  
2  
3 @contextmanager  
4 def go(num1, num2):  
5     print("go!")  
6     yield num1, num2, num1 + num2  
7  
8 with go(1, 2) as n:  
9     print(n)
```